Energy-Efficient Priority Scheduling for Task Allocation in 6G Edge Computing

Deadline-aware queues and guarded CPU slow-down on top of the EdgePlus scheduler

Keerat Malhotra Bhavya Maheshwari Bhumika Aswal

Netaji Subhas University of Technology, New Delhi, India

Abstract

Edge computing places small servers close to the devices that use them, so that results come back quickly. In a 6G smart city these servers receive a constant stream of tasks from sensors, and the tasks have very different deadlines: an air-pollution alert may need an answer within seconds, while a logging job can wait ten minutes. EdgePlus, a recent scheduler published in the *IEEE Internet of Things Journal*, uses reinforcement learning to decide which server each task should go to. Once a task reaches a server, however, EdgePlus treats all tasks alike: they wait in arrival order and run at full CPU speed.

We add two simple rules inside every server. The first is a form of priority scheduling: the server always runs the waiting task whose deadline is closest, and it drops a task as soon as that task can no longer finish in time. The second saves energy: when nothing else is waiting, the server lowers its CPU speed for a task that has time to spare, but never by more than 1.5 seconds of extra running time. We rebuilt EdgePlus in a simulator, trained it, and tested the same trained model with and without our rules on two datasets: the CityPulse data used by EdgePlus, and real sensor data collected on public buses in Delhi (AirDelhi).

At the heaviest load we tested (150,000 tasks in ten minutes), the share of tasks finished on time rises from 80.5% to 99.7% on CityPulse and from 73.0% to 96.5% on AirDelhi, and missed urgent tasks fall from 55.4% to 0.07% on CityPulse. At light load, energy per task falls by 19.8% while every task still finishes on time. The price is that relaxed tasks wait longer: the overall 95th-percentile latency at heavy load rises from 148 s to 298 s on CityPulse, although those tasks still meet their own deadlines. The paper explains each idea from first principles, with worked examples, so that readers who are new to the field can follow it.

Keywords: 6G, edge computing, task allocation, priority scheduling, earliest deadline first, energy efficiency, DVFS, reinforcement learning.

Contents

1 Introduction **3**

2 Background: the ideas you need **4**
2.1 Edge computing and 6G in plain words 5
2.2 How a server works through its tasks 6
2.3 CPU speed and energy 7
2.4 Reinforcement learning in one page 8
2.5 The statistics used in this paper 8

3 EdgePlus: the starting point **9**
3.1 The city and its servers 9
3.2 How EdgePlus picks a server 9
3.3 What EdgePlus reports, and what it leaves open 9

4 Our method: two rules inside each server **9**
4.1 Rule 1: earliest deadline first, with dropping . . . 10
4.2 Rule 2: guarded CPU slow-down . . . 10
4.3 The complete procedure . . . 11
4.4 Why the two rules fit together, and what they cost . . . 11

5 How we tested the idea **11**
5.1 The simulated city . . . 11
5.2 The two datasets . . . 12
5.3 From sensor readings to tasks . . . 13
5.4 Rebuilding EdgePlus . . . 14
5.5 Training the EdgePlus model . . . 14
5.6 The experiments . . . 15

6 Results **16**
6.1 Is our EdgePlus close to the original? . . . 16
6.2 Main result on CityPulse . . . 16
6.3 Main result on real Delhi data . . . 17
6.4 Which part of the method does what? . . . 18
6.5 What happens during an episode . . . 19
6.6 Which parts of the city benefit . . . 20
6.7 The price: relaxed tasks wait longer . . . 20
6.8 Energy savings in detail . . . 21
6.9 Are the differences real? . . . 22
6.10 Do the rules need a learned scheduler? . . . 22

7 Discussion **22**
7.1 What this means in practice . . . 22
7.2 Limitations . . . 23
7.3 Future work . . . 23

8 Conclusion **23**

A All simulation parameters **24**

1 Introduction

Picture a city with thousands of air-quality sensors on lamp posts, rooftops and buses. Every few seconds each sensor sends a reading, and every reading needs a little computing: checking whether pollution has crossed a dangerous level, updating a pollution map, or simply storing the value for later. Some of these jobs are urgent. If fine dust suddenly spikes next to a school, an alert that arrives a minute late is of little use. Other jobs are relaxed: a reading that is only being logged can wait ten minutes and nobody will notice.

Sending all of this work to a big cloud data centre far away adds delay. *Edge computing* solves part of the problem by placing smaller servers close to the sensors, for example next to the mobile base stations of a city, so that answers come back faster. Sixth-generation (6G) mobile networks, expected around 2030, are being designed to connect very large numbers of such devices with very low delay. Edge servers are still limited, though. When many tasks arrive at once, some of them have to wait, and some miss their deadlines.

A recent and strong answer to the question *"which server should each task go to?"* is EdgePlus [1], published in the *IEEE Internet of Things Journal* in 2026. EdgePlus uses reinforcement learning: a program that learns, by trial and error in a simulated city, how to pick a good server for each task. Before it chooses, EdgePlus removes every server that clearly cannot finish the task in time. The paper reports that this "feasibility-first" design meets more deadlines under heavy load than four other learning-based schedulers.

What EdgePlus does not change is what happens *after* the choice. Once a task reaches a server, EdgePlus assumes that it waits in line in arrival order (*first come, first served*) and that the server always runs at full CPU speed. When we rebuilt EdgePlus, we noticed two consequences:

1. **Urgent tasks wait behind relaxed ones.** At the heaviest load (150,000 tasks arriving in ten minutes), 55% of the urgent tasks missed their deadline, while only 0.03% of the relaxed logging tasks did. The servers had enough capacity overall; they were simply working on the wrong tasks first.

2. **Spare time is wasted as energy.** At light load almost every task finishes long before its deadline. A CPU that runs a little slower uses noticeably less energy, so finishing early at full speed wastes energy for no benefit.

This paper adds two simple rules inside every server to fix these two problems:

- **Rule 1, priority scheduling by deadline.** The server always runs the waiting task whose deadline is closest. This is called *earliest deadline first* (EDF). If a task can no longer finish in time even at full speed, the server drops it instead of letting it block everyone else.

- **Rule 2, energy saving.** When nothing else is waiting, the server may lower its CPU speed for a task that has time to spare. The slow-down may use at most 80% of the spare time and may add at most 1.5 seconds. Lowering the speed of a processor in this way is called *dynamic voltage and frequency scaling* (DVFS); we call our careful version *guarded DVFS*.

Both rules are fixed rules, not learned ones. This means they can be switched on for an EdgePlus model that has already been trained, with no extra training. It also makes the comparison very clean: in every experiment below, the trained EdgePlus model is exactly the same, and only the server rules change.

Figure 1 shows where everything sits. The blue parts belong to EdgePlus; the orange parts are what we add.

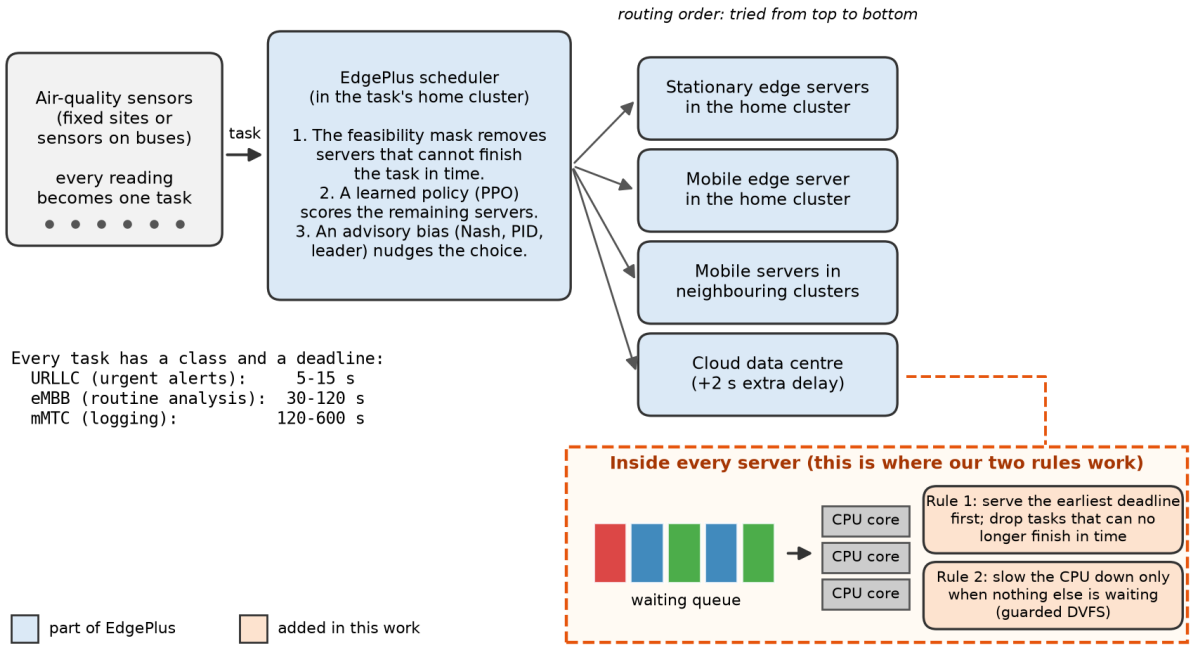


Figure 1: The system at a glance. A sensor reading becomes a task. EdgePlus (blue) decides which server should run it. Our two rules (orange) decide, inside each server, which waiting task runs next and how fast the CPU runs.

What we found. We rebuilt EdgePlus in a computer simulation of a 100 km^2 city, trained it, and tested it on two datasets: the CityPulse air-quality data used by the EdgePlus authors, and real sensor data from 13 public buses in Delhi (AirDelhi [2]). The main results are:

- At heavy load, the share of tasks finished on time rises from 80.5% to 99.7% on CityPulse and from 73.0% to 96.5% on AirDelhi. Missed urgent tasks fall from 55.4% to 0.07% on CityPulse.
- At light load, energy per task falls by 19.8% on CityPulse and 11.2% on AirDelhi, while the share of tasks finished on time stays the same or improves.
- The price is a longer wait for relaxed tasks. On CityPulse at heavy load, the 95th-percentile latency of all tasks rises from 148 s to 298 s, but the relaxed tasks still meet their deadlines (only 0.05% miss).
- Each design detail matters. Without dropping, without deadline order, with fixed class priorities instead of deadlines, or without the guard on the slow-down, results are worse.

Contributions. (1) Two server-side rules, one for priority scheduling and one for energy saving, together with a matching change to EdgePlus's waiting-time estimate. (2) An open rebuild of EdgePlus that we checked against the numbers in the original paper. (3) A careful evaluation on two datasets, one of them real measured data, with results per task class, per city area and over time, plus statistical tests.

How to read this paper. Section 2 explains all the background ideas from scratch, with small examples. Section 3 describes EdgePlus. Section 4 presents our two rules. Section 5 describes the simulated city, the data and the experiments, and Section 6 the results. Section 7 discusses what the results mean and what their limits are. Table 1 collects the terms used throughout.

2 Background: the ideas you need

This section explains, from the beginning, every idea the rest of the paper uses. Table 1 summarises the most important words; you can come back to it at any time.

Table 1: Words used in this paper.

Term	Meaning in this paper
Task	One piece of computing work created by one sensor reading, for example "check this reading for a pollution spike".
Deadline	The time by which a task must finish to be useful. A task that finishes later counts as a miss.
On time	Finished before its deadline. The *on-time rate* is the share of all tasks finished on time.
Latency	Time from the moment a task is created until it finishes.
Slack	How much time a task can still afford to lose: deadline minus the earliest possible finish time.
Edge server	A computer near the sensors. Here every edge server has several CPU cores.
Cluster	One area of the city with its own group of edge servers. Our city has 10 clusters.
Queue	The line of tasks waiting for a free CPU core on a server.
FCFS	First come, first served: tasks run in the order they arrived.
EDF	Earliest deadline first: the waiting task with the closest deadline runs next.
DVFS	Dynamic voltage and frequency scaling: running a CPU slower to save energy.
URLLC, eMBB, mMTC	The three 6G service classes, used here for urgent, routine and relaxed tasks.
Load	How many tasks arrive in one 10-minute episode: 50,000 (light), 100,000 (medium) or 150,000 (heavy).
Episode	One simulated run of 10 minutes of task arrivals, followed by the time needed to finish them.
Policy	The learned decision rule inside EdgePlus that picks a server for each task.
Feasibility mask	A filter that removes servers that cannot finish a task in time, before the policy chooses.
95th percentile	The value that 95% of measurements are below. "95th-percentile latency 10 s" means 95% of tasks finished within 10 s.

2.1 Edge computing and 6G in plain words

A *cloud data centre* is a large building full of computers, often hundreds or thousands of kilometres away from the user. It has almost unlimited capacity, but every request has to travel there and back, which adds delay. *Edge computing* places smaller computers, called edge servers, close to where the data is produced, for example at a mobile base station in each neighbourhood. They have less capacity than the cloud but answer much faster.

Each new generation of mobile networks has added new abilities. 5G introduced three kinds of service that 6G keeps and extends [3]: *ultra-reliable low-latency communication* (URLLC) for urgent messages, *enhanced mobile broadband* (eMBB) for heavier everyday traffic, and *massive machine-type communication* (mMTC) for huge numbers of small, delay-tolerant devices. EdgePlus maps air-quality tasks onto these three classes, and so do we (Table 2).

Table 2: The three task classes. Deadline ranges and shares follow EdgePlus [1]; the average CPU work is our choice (1 Gcycle = one billion CPU cycles; a 3 GHz core does three of them per second).

Class	What it stands for in our city	Deadline	Share of tasks	Average CPU work
URLLC	urgent: a reading that looks like a dangerous pollution spike	5–15 s	20%	6 Gcycles
eMBB	routine: normal analysis and map updates	30–120 s	50%	12 Gcycles
mMTC	relaxed: low readings that only need to be logged	120–600 s	30%	9 Gcycles

2.2 How a server works through its tasks

An edge server has a few CPU cores. Each core runs one task at a time from start to finish (it does not pause a task halfway to run another). When all cores are busy, new tasks wait in a *queue*. Whenever a core becomes free, the server must decide which waiting task to run next. This decision is called *scheduling*, and the rule used to make it is the *scheduling policy*. Three common ones are:

- **First come, first served (FCFS):** the task that has waited longest runs next. It is simple and fair in one sense, but it ignores deadlines completely. This is what EdgePlus assumes.
- **Fixed priority:** every task has a priority class, and the highest class always goes first. For us that would mean URLLC before eMBB before mMTC.
- **Earliest deadline first (EDF):** the waiting task whose deadline comes soonest runs next. EDF is a kind of priority scheduling in which the priority is the deadline itself, so a relaxed task gradually becomes more urgent as its deadline approaches.

EDF has a famous strength and a famous weakness. The strength: on a single processor that can pause and resume tasks, EDF meets every deadline as long as the processor is not overloaded [4], and ordering tasks by deadline also keeps the worst lateness as small as possible [5]. The weakness: when there is more work than time (*overload*), EDF can make things worse than FCFS. It keeps running the task with the closest deadline even when that task is already doomed, and every task behind it then becomes late too. This chain reaction is known as the *domino effect* [6, 7]. The usual fix is to drop, or skip, tasks that can no longer make their deadline. Our worked example below shows all three cases.

Worked example 1: why the order of tasks matters

One server with a single core receives five tasks (all times in seconds):

Task	Class	Arrives at	Needs	Deadline (relative)	Must finish by
A	mMTC	0.0	4	300	300.0
B	eMBB	0.5	3	60	60.5
C	URLLC	1.0	2	8	9.0
D	URLLC	1.5	2	6	7.5
E	URLLC	2.0	3	3	5.0

Task A arrives first and runs from 0 to 4 s under every rule, because nothing else is waiting yet. At $t = 4$ s, tasks B, C, D and E are all waiting, and the rules make different choices (Figure 2):

- **FCFS** runs them in arrival order B, C, D, E. B and C finish in time, but D finishes at 11 s (deadline 7.5) and E at 14 s (deadline 5). **3 of 5 on time.**
- **EDF without dropping** runs E first because its deadline (5 s) is the closest. But E needs 3 s and cannot finish before 7 s, so it is late anyway, and while it runs, D and C also become late. **2 of 5 on time:** the domino effect makes EDF worse than FCFS.
- **EDF with dropping (our Rule 1)** sees at $t = 4$ s that E cannot possibly finish by 5 s, drops it, and runs D (4–6 s), C (6–8 s) and B (8–11 s). **4 of 5 on time.** Task E was lost under every rule; dropping it early saved the other three.

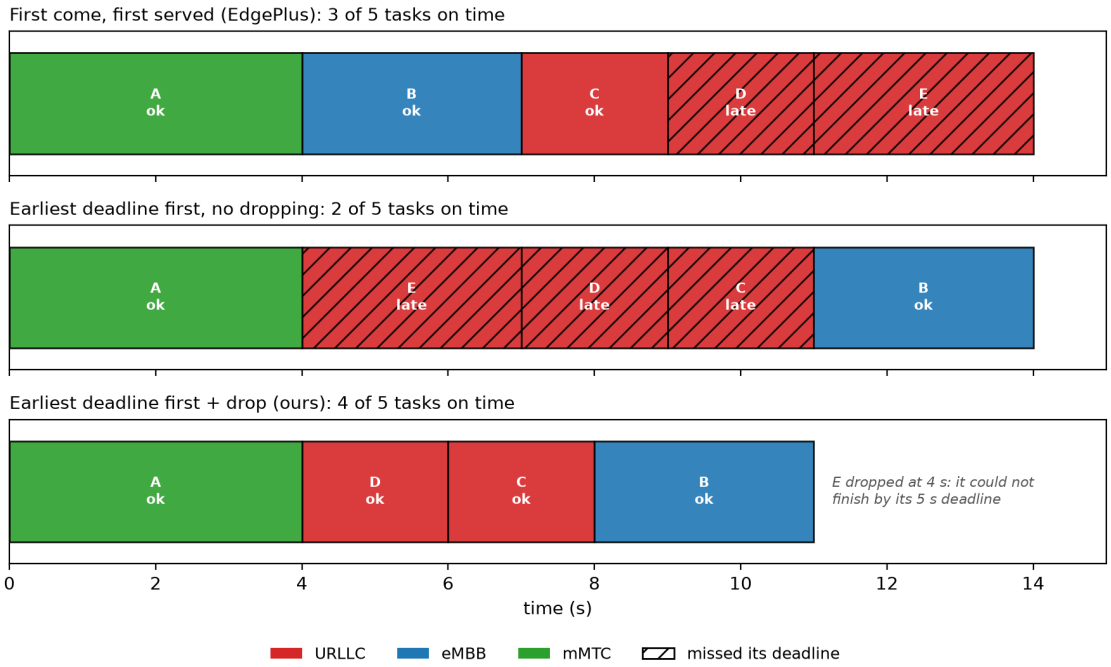


Figure 2: Worked example 1 as a timeline. Each bar is a task on the server's single core; hatched bars missed their deadline.

2.3 CPU speed and energy

A processor's power use has two parts. The *dynamic* part is spent on the switching of transistors while computing. If the clock frequency and the supply voltage are lowered together, dynamic power falls roughly with the cube of the frequency, so the dynamic energy needed for a fixed amount of work falls roughly with its square. The *static* part is power drawn just because the chip is switched on and busy; it adds up with running time, so running slower makes it larger. DVFS, lowering the voltage and frequency when full speed is not needed, exploits this trade-off and is widely used in phones and servers [8, 9].

We model the energy of a task run at a fraction ϕ of full speed ($0 < \phi \le 1$) as

$$E(\phi) = E_{\text{full}} \left(0.75\,\phi^2 + \frac{0.25}{\phi} \right), \tag{1}$$

where E_{full} is its energy at full speed, 75% of which is dynamic and 25% static. At $\phi = 1$ the formula gives exactly E_{full}, so the energy of EdgePlus itself is unchanged. Figure 3 (left) plots the formula. Slowing down saves energy until about 55% speed; below that, the extra static energy outweighs the dynamic saving. Table 3 lists the speeds our rule may choose.

Table 3: The CPU speeds our rule may choose, how much longer a task takes, and how much energy it then needs (from Equation 1).

Speed (fraction of full)	100%	90%	80%	70%	60%	50%
Running time	1.00×	1.11×	1.25×	1.43×	1.67×	2.00×
Energy needed	1.000	0.885	0.793	0.725	0.687	0.688
Energy saved	0%	11.5%	20.8%	27.5%	31.3%	31.2%

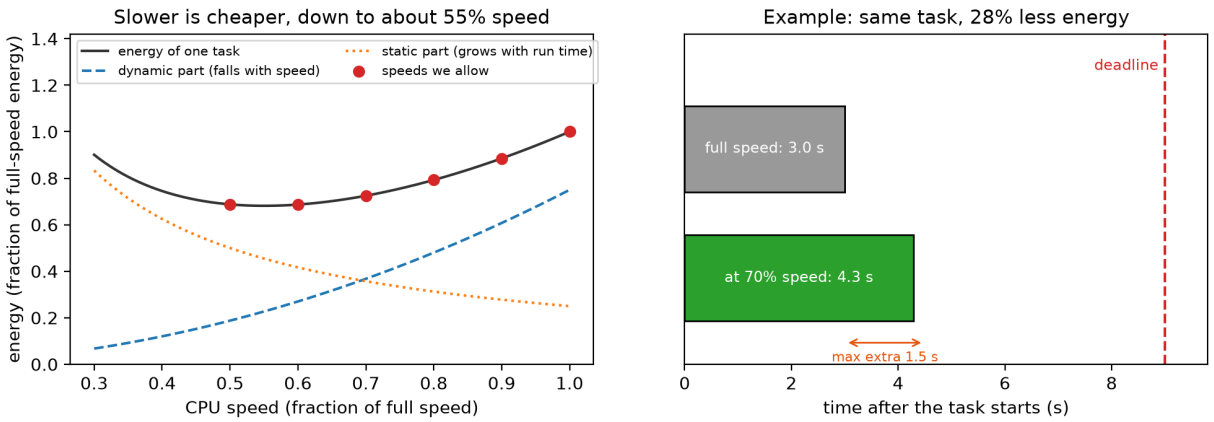


Figure 3: Left: energy of one task against CPU speed (Equation 1); the dots are the speeds we allow. Right: worked example 2. A task needing 3 s at full speed with 6 s of spare time is run at 70% speed, takes 4.3 s and uses 28% less energy.

There is a catch. A slowed-down task holds its CPU core for longer. If an urgent task arrives in the meantime and all cores are busy, the urgent task has to wait for that longer time. This is why our energy rule comes with a guard (Section 4.2).

2.4 Reinforcement learning in one page

Reinforcement learning (RL) is a way for a program, called the *agent*, to learn good decisions by trial and error. At each step the agent looks at the current situation (the *state*), picks an *action*, and later receives a *reward*: a number that says how good the outcome was. Over many attempts the agent adjusts its *policy*, the rule that maps states to actions, so that it collects more reward.

In EdgePlus, a step happens every time a new task arrives:

- the **state** describes the task (class, deadline, size) and the candidate servers (how busy they are, how long the task would wait, how far away they are);
- the **action** is the choice of one server;
- the **reward** is positive if the task finishes on time and negative if it is late, with smaller penalties for long latency, high energy, using the cloud and choosing unreliable servers.

EdgePlus learns with *proximal policy optimization* (PPO) [10], a popular RL method that improves the policy in small, careful steps so that learning does not become unstable. PPO uses two neural networks: the *actor*, which gives each candidate server a score, and the *critic*, which estimates how good the current situation is and helps the actor tell lucky outcomes from good decisions.

In simple words: the feasibility mask

Before the actor scores the servers, EdgePlus removes every server that clearly cannot finish the task in time (or that is out of memory, battery or reserved capacity). The actor then only chooses among the rest. This is like a delivery app that hides restaurants which are already closed before you start choosing: you can still make a poor choice, but never an impossible one. The EdgePlus authors found that this mask was the single most important part of their design.

2.5 The statistics used in this paper

Every number we report is an average over 30 runs: 3 separately trained EdgePlus models × 10 different test episodes. To show how certain an average is, we give a *95% confidence interval* (CI), written "mean ± half-width"; roughly, it is the range in which the true average very likely lies. To check whether a difference is real or just luck, we use *paired* comparisons: the same trained model on the same test episode is run

once with EdgePlus's rules and once with ours, and only the difference matters. The *Wilcoxon signed-rank test* [11] turns these 30 differences into a *p-value*, the probability of seeing differences this consistent if the rules made no difference at all; small values (below 0.05) mean the difference is real. Because we run many tests, we adjust the p-values with the *Benjamini–Hochberg* method [12], which keeps the share of false alarms low.

3 EdgePlus: the starting point

3.1 The city and its servers

EdgePlus [1] models a smart city of 100 km^2 divided into 10 clusters. Each cluster has stationary edge servers and mobile edge servers (servers mounted on vehicles that can drive to busy areas), and the whole city shares a cloud data centre. A task is first offered to servers in a fixed *routing order*: the stationary servers of its own cluster, then the mobile server of its own cluster, then mobile servers in neighbouring clusters, and finally the cloud. The latency of a task is the sum of four parts: sending the data (transmission), driving a mobile server closer if needed (movement), waiting in the queue, and computing.

3.2 How EdgePlus picks a server

For every task, EdgePlus does three things:

1. The **feasibility mask** removes servers that cannot finish the task in time or that would break a limit: memory, battery of a mobile server, or the capacity reserved for each class ("network slices").
2. The **actor network** gives every remaining server a score.
3. An **advisory layer** adds small bonuses to the scores. A *Nash sharing* rule favours clusters willing to accept extra work, a *PID controller* (a classic feedback controller) steers each cluster towards a target utilisation, and *leader election* favours the least-loaded, best-charged server of each cluster.

The task goes to a server picked according to the final scores. EdgePlus also has a *blockchain-lite ledger*: it records completed tasks, and the record is used to give each server a *reputation* (its recent share of on-time tasks) and each cluster a *debt* (how far it is below a throughput target). Both enter the state, the reward and the mask; urgent tasks avoid servers with poor reputation.

3.3 What EdgePlus reports, and what it leaves open

EdgePlus also includes a particle swarm optimisation (PSO) outer loop that tunes training, and a security layer (encryption and signatures). At the heaviest load (150,000 tasks) the paper reports 82.77% of tasks on time, a median latency of 53.2 s and a 95th-percentile latency of 147.1 s; without PSO the on-time rate is 82.0%.

Three things are left open. First, the queue model that EdgePlus uses to estimate waiting time (Kingman's formula [13]) assumes FCFS; nothing gives urgent tasks priority inside a server. The paper does attach a high/moderate/low priority label to each task, but it is only used to break ties. Second, compute energy is modelled as a constant times the CPU work, so the CPU always runs at full speed. Third, the paper defines a URLLC reliability measure but reports results only for all tasks together, so it cannot show which classes miss their deadlines. Our work addresses all three.

4 Our method: two rules inside each server

Both rules work inside a server and use the same idea: *slack*, the time a task can still afford to lose. If a task with deadline D would finish at time $t + w$ when started now at full speed (it needs w seconds), its slack is

$$\sigma = D - (t + w). \tag{2}$$

A task with small slack is urgent and should go first at full speed. A task with large slack can wait, or can run more slowly.

4.1 Rule 1: earliest deadline first, with dropping

Whenever a CPU core becomes free, the server:

1. looks at all waiting tasks and takes the one with the earliest deadline;
2. checks whether it can still finish in time at full speed, that is, whether its slack $\sigma \geq 0$;
3. if it can, runs it; if it cannot, drops it (it counts as a missed deadline) and goes back to step 1.

Dropping only removes tasks that are already lost: such a task would have missed its deadline under any rule. The benefit is that the CPU time it would have used goes to tasks that can still be saved, which stops the domino effect of worked example 1.

Keeping the mask honest. EdgePlus's mask asks "how long would this task wait at this server?". With FCFS the answer is "until all the work already queued has been done". With EDF, a new task only waits for queued tasks whose deadline is earlier than its own. So we change the estimate to

$$\widehat{W} = r_{\min} + \frac{1}{c}\Big(\sum_{\text{queued } j \text{ with } D_j \leq D} w_j \;+\; w_{\text{in transit}} \Big), \tag{3}$$

where c is the number of cores, $r_{\min}$ is the time until the first busy core becomes free (zero if a core is free), w_j is the work of a queued task j, and $w_{\text{in transit}}$ is work already sent to this server but not yet arrived. Without this change the mask would keep rejecting busy servers for urgent tasks, even though EDF would let those tasks jump the queue and finish in time.

Worked example 3: the waiting-time estimate

A server has 2 cores, both busy for another 1 s. In its queue are a relaxed task (deadline in 400 s, needs 6 s) and a routine task (deadline in 80 s, needs 4 s). A new urgent task arrives with a 10 s deadline. Under FCFS it would wait $1 + (6+4)/2 = 6$ s. Under EDF neither queued task has an earlier deadline, so it waits only $r_{\min} = 1$ s. With the FCFS estimate the mask might wrongly reject this server; with Equation 3 it correctly accepts it.

4.2 Rule 2: guarded CPU slow-down

When a task is about to start, the server may run it below full speed, but only if all three conditions hold:

1. **nothing else is waiting** for this server and nothing is on its way to it;
2. the extra running time is at most **80% of the task's slack**, so it still finishes on time with a margin;
3. the extra running time is at most **1.5 seconds** (the guard).

Among the allowed speeds $\{50\%, 60\%, \ldots, 100\%\}$ the server picks the lowest one that satisfies these conditions. Written as a formula, it picks the smallest ϕ with

$$\frac{w}{\phi} - w \;\leq\; \Delta, \qquad \Delta = \begin{cases} \min(0.8\,\sigma,\; 1.5\,\text{s}) & \text{if nothing is waiting,} \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

Why the guard? Conditions 2 and 3 protect the task itself and the tasks that might arrive while it runs. Without condition 1 and the 1.5 s cap, a relaxed task with 300 s of slack could be slowed to half speed, doubling its running time. An urgent task arriving a moment later could then find every core occupied by slow tasks. We test exactly this "unguarded" version in Section 6.4 and it performs very badly.

Worked example 2: slowing down safely

A task needs 3 s at full speed and has 6 s of slack, and nothing else is waiting. The allowed extra time is $\min(0.8 \times 6,\ 1.5) = 1.5$ s. At 70% speed the task takes $3/0.7 = 4.3$ s, only 1.3 s extra, so 70% is allowed; 60% would take 5.0 s, 2 s extra, which is too much. So the server runs it at 70% speed. From Table 3, it needs 0.725 of the full-speed energy: **27.5% less energy**, and it still finishes 4.7 s before its deadline (Figure 3, right).

4.3 The complete procedure

Algorithm 1 combines the two rules. It runs every time a core of a server becomes free, and also when a task reaches a server that has a free core.

Algorithm 1 What a server does when a CPU core becomes free at time t

while the queue is not empty **do**
  take the waiting task i with the earliest deadline D_i ▷ Rule 1: EDF
  $\sigma \leftarrow D_i - t - w_i$ ▷ slack at full speed
  if $\sigma < 0$ **then**
    drop task i (count it as missed) and **continue** ▷ Rule 1: drop
  end if
  if the queue is empty and nothing is in transit **then** $\Delta \leftarrow \min(0.8\sigma, 1.5\,\text{s})$
  else $\Delta \leftarrow 0$
  end if
  $\phi \leftarrow$ the lowest allowed speed with $w_i/\phi - w_i \le \Delta$ ▷ Rule 2: guarded DVFS
  run task i on the free core at speed ϕ and **stop**
end while

4.4 Why the two rules fit together, and what they cost

Both rules follow the same logic: urgent tasks (small slack) go first and run at full speed; relaxed tasks (large slack) wait or run slower. Rule 1 improves the on-time rate when servers are busy, which is exactly when Rule 2 rarely acts because queues are rarely empty. When servers are quiet, Rule 1 changes little and Rule 2 saves energy. So the two rules help at different loads without getting in each other's way.

Neither rule needs training. The extra work per decision is small: sorting the queue by deadline and checking a few speeds. In our simulator, a whole episode including the simulation itself takes 0.23 ms per task with EdgePlus's rules and 0.30 ms with ours, on one CPU core.

5 How we tested the idea

5.1 The simulated city

We wrote a simulator in Python that follows EdgePlus's description: a 10 km × 10 km city with 10 clusters, each with 4 stationary edge servers and 1 mobile edge server, plus one cloud. The simulator is *event driven*: it jumps from one event to the next (a task arrives, a task finishes, a mobile server reaches a new area), so it can process 150,000 tasks in under a minute. Table 4 lists the servers.

Table 4: The servers in the simulated city.

Server type	How many	Details
Stationary edge server	4 per cluster (40)	24 cores each, at 2.4, 3.0, 3.6 and 3.0 GHz; 64 GB memory; placed around the cluster centre
Mobile edge server	1 per cluster (10)	16 cores at 2.4 GHz; 32 GB memory; drives at 15 m/s; 2 km radio range; 2 kJ battery; can move to an overloaded neighbouring cluster every 20 s
Cloud data centre	1	32 cores at 3.5 GHz; 2 s extra delay for sending data there and back

How do we know where the clusters are? Like EdgePlus, we group the sensor locations with *weighted K-means*, a standard method that places 10 centres so that every sensor is close to one of them; sensors with higher pollution readings count a little more. Clusters whose centres are next to each other (found with a Delaunay triangulation) are *neighbours* and may share their mobile servers. Figure 4 shows the result for both datasets.

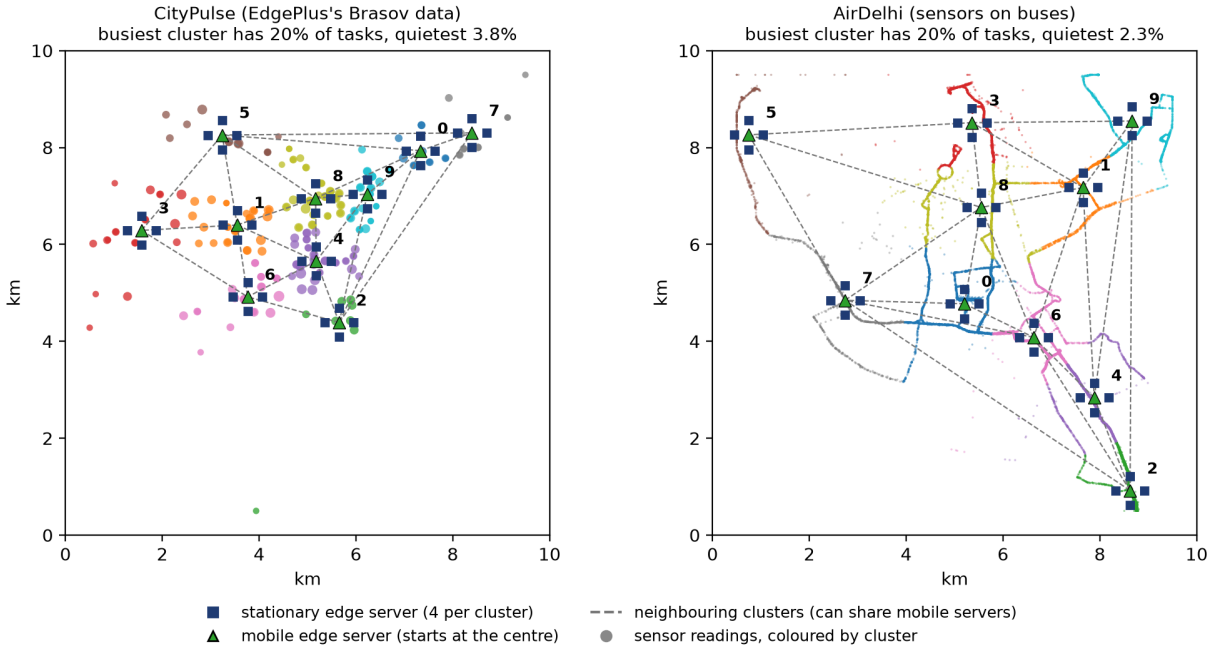


Figure 4: The two simulated cities. Coloured dots are sensor locations (left) or bus positions (right), coloured by cluster; numbers are cluster IDs; dashed lines join neighbouring clusters. Both cities use the same server inventory, but tasks are spread very unevenly: the busiest cluster receives about 20% of all tasks in both cities, and the quietest only 2–4%.

5.2 The two datasets

Table 5 compares the two datasets.

CityPulse [14] is the dataset that EdgePlus calls the Braşov air-pollution traces: 449 sensor sites reporting five pollution values every five minutes from 1 August to 1 October 2014, 7.89 million readings in total. Two details are worth knowing. The download file on the CityPulse website is named after the city of Aarhus, and CityPulse describes the values as *generated*: they follow a bounded random walk rather than real measurements. Also, all sites report at the same moments; we spread each site's reports over its five-minute interval, and tasks then arrive at a perfectly steady rate.

AirDelhi [2] is real measured data, released by IIT Delhi in 2023: low-cost particulate matter (PM_1, $PM_{2.5}$, PM_{10}) sensors mounted on 13 public buses, recording every 3 seconds together with GPS position. We use December 2020 and January 2021. We removed readings with impossible positions or values, and

readings taken while a bus was standing still (it moved less than 50 m in about 3 minutes). Without this step, 38% of all readings came from just three 1 km^2 squares, mostly from buses standing still, probably at depots. This left 4.04 million readings. Because buses move between areas and their activity changes through the day, AirDelhi tasks arrive in strong bursts (Figure 5).

Table 5: The two datasets.

	CityPulse (EdgePlus's "Braşov")	AirDelhi
Source	CityPulse project, University of Surrey	IIT Delhi, NeurIPS 2023 Datasets track
Sensors	449 fixed sites (208 distinct locations)	13 sensors on public buses
Period used	1 Aug – 1 Oct 2014	1 Dec 2020 – 30 Jan 2021
Values	5 pollution indices (ozone, particles, CO, SO_2, NO_2)	$PM_{2.5}$ and PM_{10} concentrations
Real or generated	generated (bounded random walk)	real measurements
Readings used	7.89 million	4.04 million (after removing parked-bus readings)
Training / test split	first 43 days / last 18 days	December / January
Arrival pattern	perfectly steady	bursty: from 6 to 455 tasks per second at heavy load

5.3 From sensor readings to tasks

Following EdgePlus's labelling procedure, every reading becomes one task:

1. **Severity.** For each pollutant we measure how unusual the reading is compared with the training period (a robust z-score), and how fast it is rising. The severity of the reading is its most unusual pollutant plus half of its fastest rise.
2. **Class.** Within each episode, the 20% most severe readings become URLLC tasks, the next 50% eMBB and the remaining 30% mMTC, matching EdgePlus.
3. **Deadline.** Drawn at random from the class's range in Table 2.
4. **Size.** The CPU work is drawn around the class average, a little higher for more severe readings.
5. **Home cluster.** The cluster whose centre is closest to the sensor (or to the bus at that moment).
6. **Arrival time.** An episode takes N consecutive readings and squeezes their real time span into a 10-minute window. N is 50,000, 100,000 or 150,000, giving on average 83, 167 or 250 new tasks per second. At 150,000 tasks an episode covers about 28 hours of CityPulse data or 43 hours of AirDelhi data.

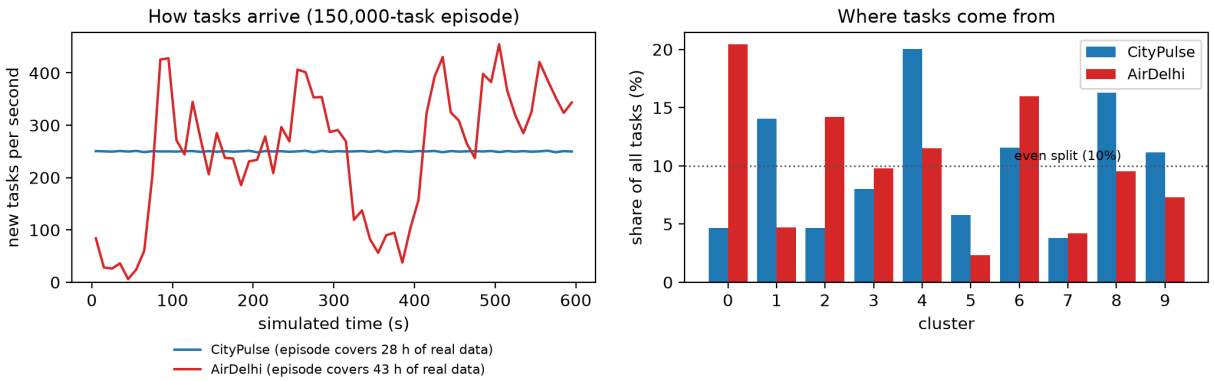


Figure 5: The workload. Left: new tasks per second during one heavy-load test episode; CityPulse is perfectly steady, AirDelhi is bursty. Right: share of all tasks coming from each cluster.

5.4 Rebuilding EdgePlus

We could not find public code for EdgePlus, so we rebuilt it from the paper. Table 6 shows what we kept and what we simplified. Two simplifications matter most. We left out PSO, so our baseline corresponds to the paper's "EdgePlus without PSO" variant. And we left out the encryption and blockchain machinery, but kept the reputation and debt signals that the ledger feeds into the scheduler; the ledger's own check takes 0.39 ms according to the paper and happens after a task finishes, so it does not affect latency. All other parameter values are listed in Appendix A.

Table 6: What we rebuilt from EdgePlus. Everything here applies equally to the baseline and to our rules.

EdgePlus part	In our rebuild
Feasibility mask, routing order, slices	Kept: deadline, memory, battery, slice budget and URLLC reputation checks
PPO actor and centralised critic	Kept: one actor network shared by all clusters that scores each candidate server; a critic that sees the whole city
Advisory layer (Nash, PID, leader)	Kept, with EdgePlus's PID settings (1.90, 1.20, 0.06)
Reward	Kept: on-time bonus, late penalty, latency, energy, cloud, reputation and debt terms
Reputation and debt from the ledger	Kept as signals; the blockchain itself is not simulated
Waiting-time estimate	Uses the server's real queue instead of Kingman's formula (more accurate for the baseline too)
Servers	Multi-core servers, simulated task by task
Mobile server paths	Straight line × 1.3 (instead of CBS–D* Lite path planning)
PSO outer loop	Not included (baseline = EdgePlus without PSO)
Encryption, signatures, security tests	Not included (no effect on scheduling)

We calibrated one number: the number of cores per stationary server. We chose 24 because the trained EdgePlus model then comes close to the paper's heavy-load result (80.5% on time against 82.0%), while every task is on time at light load, as in the paper.

5.5 Training the EdgePlus model

We trained one EdgePlus model for each load and each of 3 random seeds (9 models). Each model trained for 60 rounds; in each round it simulated one full training episode, collected one decision per task, and updated its networks with PPO. Training took 25, 45 and 61 minutes per model on one CPU core at the three loads, 6.6 CPU-hours in total. No graphics card was needed, because the networks are small and the simulator does most of the work.

Figure 6 shows learning. At light and medium load the model quickly reaches almost 100% on time. At heavy load the on-time rate during training climbs from about 74.6% in the first ten rounds to 79.4% in the last ten. (During training the model tries random choices on purpose, so its score is lower than in the tests, where it always takes its best choice.) The spikes come from training episodes that happened to be easier. As a check that learning helped, the trained model beats a simple non-learning scheduler at heavy load by 8.5 points (80.5% against 72.0% on time).

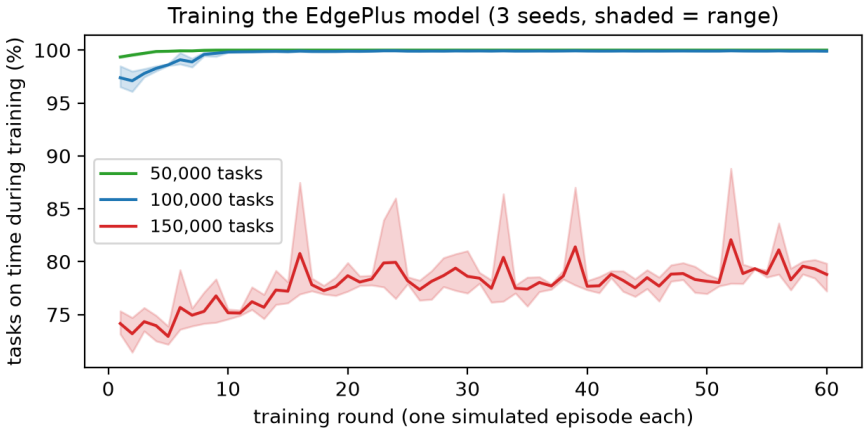


Figure 6: Training the EdgePlus model: share of tasks on time in each training round (line: average of 3 seeds; shaded: range).

5.6 The experiments

Each of the 9 trained models was tested on 10 test episodes taken from the test period of the data, giving 30 runs per setting. In every test we ran the same model with the different server rules listed in Table 7. For AirDelhi we used the models trained on CityPulse without retraining them, which also tests how well everything carries over to a new city with real data. Table 8 explains the measurements.

Table 7: The server rule sets we compared. The first is EdgePlus as published; the last is our full method.

Name	Queue order	Late tasks	CPU speed
EdgePlus (baseline)	FCFS	kept	always full speed
Drop late tasks only	FCFS	dropped	always full speed
Deadline first only	EDF	kept	always full speed
Fixed class priority + drop	URLLC, then eMBB, then mMTC	dropped	always full speed
Rule 1: deadline first + drop	EDF	dropped	always full speed
Unguarded slow-down	FCFS	kept	slowed whenever the task has slack
Rule 2: guarded slow-down	FCFS	kept	guarded DVFS (Section 4.2)
Both rules (ours)	EDF	dropped	guarded DVFS

Table 8: What we measure.

Measurement	Meaning
On time (%)	Share of all tasks that finished before their deadline. Dropped tasks count as missed.
Missed (%) per class	Share of URLLC, eMBB or mMTC tasks that missed their deadline.
Median latency	Half of the finished tasks took less than this.
95th-percentile latency	95% of the finished tasks took less than this; a measure of the slowest tasks.
Energy per task	Average energy used per task, in millijoules (mJ), including transmission and computing.
Cloud share	Share of tasks sent to the cloud.
Dropped (%)	Share of tasks dropped by Rule 1 (already included in “missed”).

6 Results

6.1 Is our EdgePlus close to the original?

Before improving EdgePlus we checked that our rebuild behaves like the original (Table 9). At light load both reach 100% on time with latencies of a few seconds. At heavy load ours reaches 80.5% on time against 82.0% for the published EdgePlus without PSO, with nearly the same 95th-percentile latency (148 s against 147–159 s). Our median latency and cloud share are lower. The one clear difference is the medium load, where the paper reports 82.5% and we measure 99.9%. The steady arrivals of the CityPulse data leave the servers far from full at medium load in our simulator. AirDelhi, whose arrivals are bursty, does produce a congested medium load, so it covers this case.

Table 9: Our rebuilt EdgePlus against the published numbers (CityPulse data).

Load	Source	On time (%)	Median (s)	95th pct (s)	Cloud (%)
50,000	EdgePlus paper	100.00	3.98	10.24	–
	our rebuild	100.00	3.55	7.35	0.00
100,000	EdgePlus paper	82.50	–	about 151	–
	our rebuild	99.87	4.20	11.79	0.09
150,000	EdgePlus paper	82.77	53.23	147.09	6.1
	EdgePlus paper, without PSO	82.0	54.8	158.9	6.6
	our rebuild (without PSO)	80.52	27.95	148.10	3.78
	simple scheduler, no learning	71.99	38.39	125.01	3.79

6.2 Main result on CityPulse

Table 10 and Figure 7 compare EdgePlus with our rules on the dataset used by the EdgePlus authors.

At **heavy load**, Rule 1 raises the share of tasks on time from 80.5% to 99.6%. Missed URLLC tasks fall from 55.4% to 0.07% and missed eMBB tasks from 16.8% to 0.65%, while mMTC tasks stay almost perfect (0.05% missed). Only 0.36% of tasks are dropped. The median latency drops from 28 s to under 6 s. In other words, the servers had enough capacity all along; under FCFS they were spending it on the wrong tasks first.

At **light load**, every task is on time with or without our rules, so Rule 1 has nothing to fix. Here Rule 2 does the work: energy per task falls from 27.04 mJ to 21.69 mJ, a saving of 19.8%. The saving shrinks as load grows (10.0% at medium, 5.2% at heavy load) because busy servers rarely have an empty queue, which is when Rule 2 is allowed to act. The cost at light load is about one extra second of latency (median 3.6 s to 4.7 s), well within every deadline.

Table 10: CityPulse: EdgePlus with and without our rules (average of 30 runs; the 95% confidence intervals are small, for example ±0.50 for EdgePlus's 80.52% and ±0.10 for our 99.65%).

Load	Rules	On time (%)	Missed (%) URLLC	Missed (%) eMBB	Missed (%) mMTC	Median latency (s)	95th pct latency (s)	Energy (mJ/task)
50,000	EdgePlus	100.00	0.00	0.00	0.00	3.55	7.35	27.04
	+ Rule 1	100.00	0.00	0.00	0.00	3.55	7.35	27.04
	+ Rule 2	99.99	0.04	0.00	0.00	4.73	8.24	21.76
	+ both (ours)	100.00	0.01	0.00	0.00	4.72	8.19	**21.69**
100,000	EdgePlus	99.87	0.66	0.00	0.00	4.20	11.79	27.61
	+ Rule 1	99.99	0.04	0.00	0.00	3.82	9.72	27.59
	+ Rule 2	99.84	0.79	0.00	0.00	5.05	12.39	24.83
	+ both (ours)	**99.99**	0.04	0.00	0.00	4.59	10.48	**24.86**
150,000	EdgePlus	80.52	55.36	16.80	0.03	27.95	148.10	28.36
	+ Rule 1	99.64	0.07	0.65	0.05	5.46	290.63	28.07
	+ Rule 2	79.60	57.11	17.94	0.04	29.81	155.12	27.11
	+ both (ours)	**99.65**	0.07	0.64	0.05	5.99	298.37	**26.88**

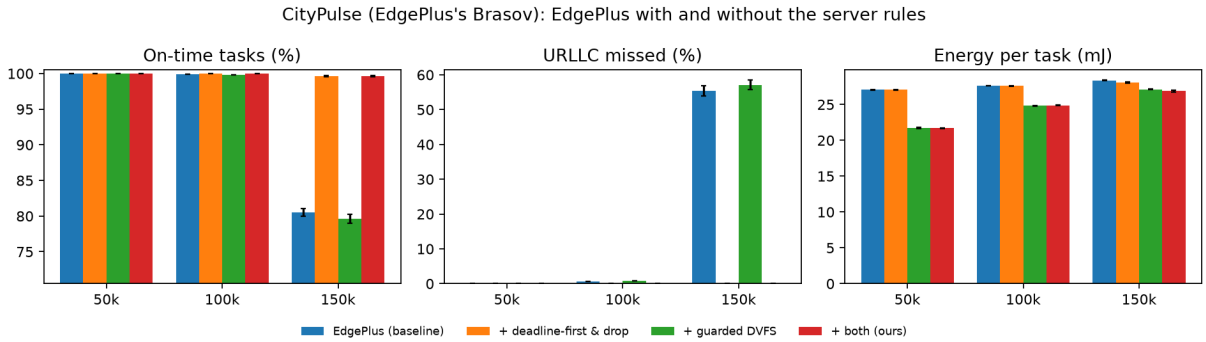


Figure 7: CityPulse: share of tasks on time, missed URLLC tasks and energy per task at the three loads. Error bars are 95% confidence intervals.

6.3 Main result on real Delhi data

AirDelhi is a harder test (Table 11, Figure 8). Its bursts overload some clusters for minutes at a time, and the EdgePlus models were trained on CityPulse and never saw Delhi. EdgePlus finishes 98.8%, 89.3% and 73.0% of tasks on time at the three loads. With both rules the same models reach 99.8%, 98.8% and 96.5%. Missed URLLC tasks fall from 38.6% to 0.97% at medium load and from 73.0% to 3.7% at heavy load, and energy per task falls by 11.2%, 5.5% and 7.3%. Our rules contain nothing learned, so they carry over to a new city unchanged.

Table 11: AirDelhi (models trained on CityPulse, not retrained): EdgePlus with and without our rules (average of 30 runs).

Load	Rules	On time (%)	Missed (%) URLLC	eMBB	mMTC	Median latency (s)	95th pct latency (s)	Energy (mJ/task)
50,000	EdgePlus	98.75	5.76	0.19	0.00	4.38	25.26	24.03
	+ Rule 1	99.85	0.16	0.24	0.00	3.71	26.74	23.89
	+ Rule 2	98.58	6.54	0.22	0.00	5.42	26.21	21.61
	+ both (ours)	**99.84**	0.17	0.25	0.00	4.66	28.37	**21.33**
100,000	EdgePlus	89.26	38.58	5.98	0.11	13.28	109.55	24.60
	+ Rule 1	98.74	0.95	1.55	0.99	6.39	153.86	23.96
	+ Rule 2	88.84	40.57	6.03	0.13	14.07	109.86	23.87
	+ both (ours)	**98.78**	0.97	1.50	0.94	7.10	154.95	**23.24**
150,000	EdgePlus	72.97	72.96	24.43	0.76	38.99	338.80	25.04
	+ Rule 1	96.54	3.55	3.97	2.57	15.75	316.50	23.56
	+ Rule 2	72.35	74.97	24.84	0.80	39.91	341.52	24.76
	+ both (ours)	**96.47**	3.68	4.04	2.59	16.86	316.13	**23.21**

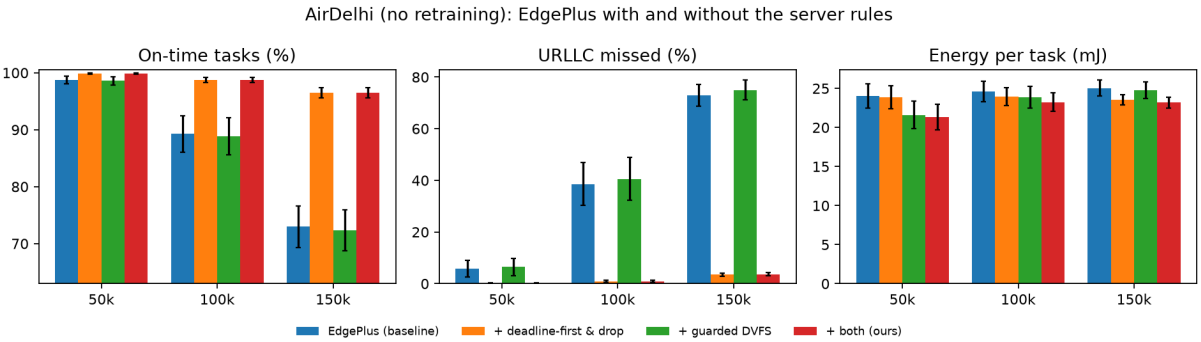


Figure 8: AirDelhi (models trained on CityPulse): share of tasks on time, missed URLLC tasks and energy per task. Error bars are 95% confidence intervals.

6.4 Which part of the method does what?

To see why the method works, we switched its parts on and off one at a time at the heaviest CityPulse load (Table 12, Figure 9).

- **Dropping alone** (still FCFS) raises the on-time share to 91.2%, but 40.3% of URLLC tasks still miss and 8.8% of all tasks are dropped. Without deadline order, urgent tasks reach the front of the queue too late and are then dropped.
- **Deadline order alone** (no dropping) reaches 97.4%. In the full city we did not see the dramatic domino effect of worked example 1, probably because EdgePlus's mask already steers new tasks away from servers with long queues. Still, without dropping, 4.2% of eMBB and 1.4% of mMTC tasks miss.
- **Fixed class priority** (URLLC first, then eMBB, then mMTC, with dropping) protects URLLC tasks (0.11% missed) but leaves 10.1% of mMTC tasks late: a relaxed task never gets ahead of an urgent one, even when its own deadline is about to pass. Deadlines make a better priority than fixed classes.
- **Deadline order with dropping** (Rule 1) gives the best on-time share, 99.6%.

- **Unguarded slow-down** is a disaster: only 46.0% on time at heavy load (and 66.0% at medium load), because slowed tasks block cores that urgent tasks need. It saves the most energy at light load (28.1%), but at too high a price.
- **Guarded slow-down** (Rule 2) keeps the on-time share within one point of EdgePlus at every load, on both datasets, while saving energy.

Table 12: Switching the parts of the method on and off (CityPulse, 150,000 tasks, average of 30 runs).

	On time	Missed (%)			95th pct	Energy
Rules	(%)	URLLC	eMBB	mMTC	latency (s)	(mJ/task)
EdgePlus (FCFS, keep late tasks)	80.52	55.36	16.80	0.03	148.1	28.36
Drop late tasks only	91.24	40.33	1.39	0.00	86.7	26.46
Deadline first only	97.41	0.32	4.24	1.37	298.0	28.29
Fixed class priority + drop	96.49	0.11	0.89	10.14	171.0	27.35
Rule 1: deadline first + drop	99.64	0.07	0.65	0.05	290.6	28.07
Unguarded slow-down	45.96	80.11	67.55	14.16	637.0	24.48
Rule 2: guarded slow-down	79.60	57.11	17.94	0.04	155.1	27.11
Both rules (ours)	**99.65**	0.07	0.64	0.05	298.4	26.88

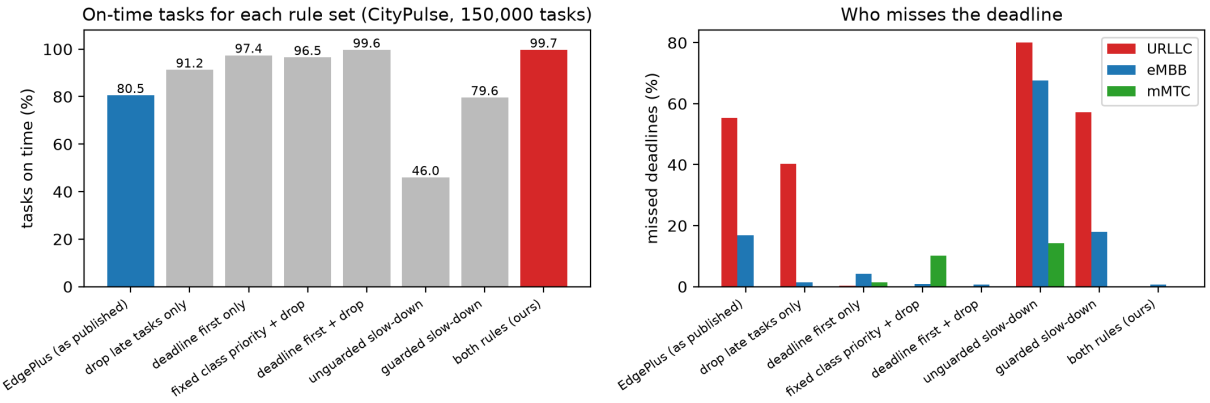


Figure 9: The parts of the method, CityPulse at 150,000 tasks. Left: share of tasks on time. Right: missed deadlines per class.

6.5 What happens during an episode

Figure 10 follows one heavy-load test episode over its 10 minutes and shows the on-time share of tasks grouped by when they arrived. On CityPulse, EdgePlus starts well, but its queues grow steadily in the busy clusters, and by the end of the window only about 60% of new tasks finish on time. With our rules the share stays near 99% throughout. On AirDelhi, EdgePlus collapses during each burst, down to about 9% at the worst moment, and recovers in between. With our rules the bursts still hurt, but the on-time share stays above 83% at all times. In this particular episode the totals are 60.7% for EdgePlus and 93.9% for our rules.

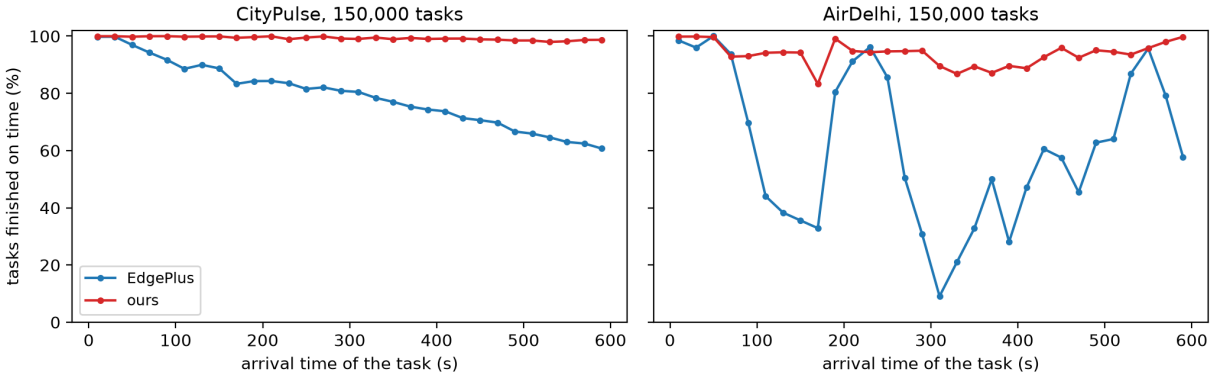


Figure 10: On-time share of tasks by arrival time (20-second groups) in one heavy-load test episode.

6.6 Which parts of the city benefit

Figure 11 breaks the same episodes down by cluster. Quiet clusters were already fine. The gains are in the busy clusters. On CityPulse, cluster 4, which receives 20% of all tasks, goes from 60.0% to 98.0% on time, and cluster 8 from 66.4% to 99.2%. On AirDelhi the busiest clusters 0, 2 and 6 go from about 45% to 88–94%. The city as a whole also becomes fairer: no area is left far behind.

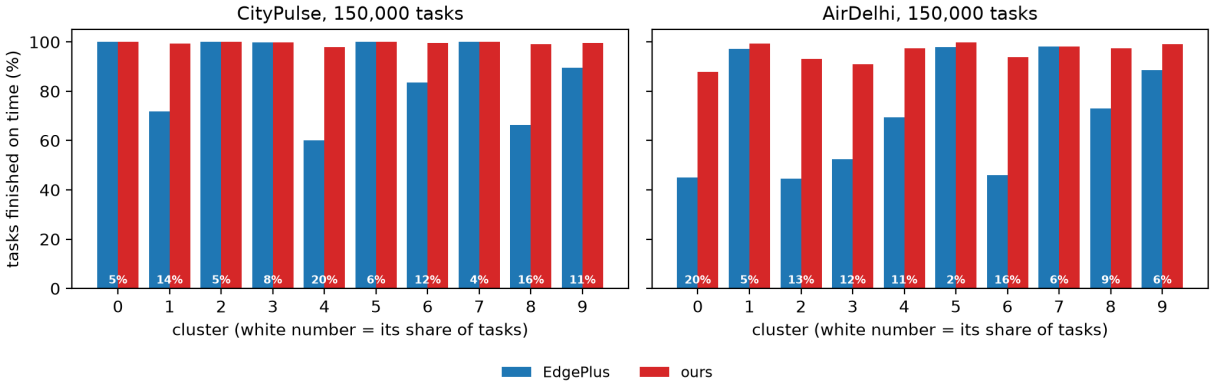


Figure 11: On-time share per cluster in one heavy-load test episode. The white number on each bar is the cluster's share of all tasks.

6.7 The price: relaxed tasks wait longer

Our rules are not free. Deadline order moves waiting time from urgent tasks to relaxed ones (Figure 12). On CityPulse at heavy load, the 95th-percentile latency of URLLC tasks falls from 92 s to 6 s and that of eMBB tasks from 105 s to 87 s, but that of mMTC tasks rises from 223 s to 439 s. Because mMTC tasks are 30% of all tasks, the 95th-percentile latency of all tasks together rises from 148 s to 298 s. These relaxed tasks still meet their deadlines of 120–600 s: only 0.05% of them miss.

Figure 13 shows the full picture for each class. With our rules almost every URLLC task is done within its 5–15 s window, and far more eMBB tasks finish early, while mMTC tasks are deliberately pushed later, though still inside their window. On AirDelhi the overall 95th-percentile latency at heavy load does not change significantly (339 s against 316 s), and at medium load it rises from 110 s to 155 s.

The lesson is that a single latency number over all classes can be misleading when deadlines range from 5 to 600 seconds. A system that finishes a logging task in 400 seconds instead of 200 has lost nothing if the deadline is 600 seconds. Latency is best reported per class, next to the share of deadlines met.

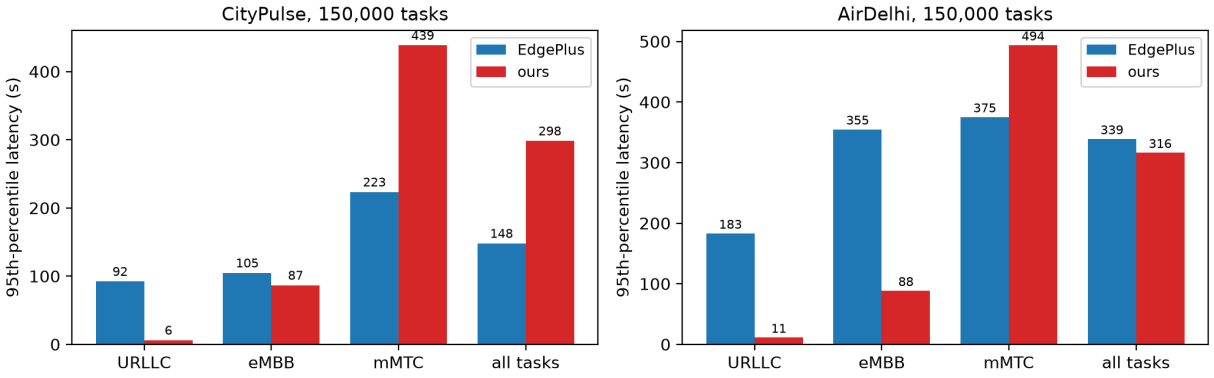


Figure 12: 95th-percentile latency by class at heavy load (average of 30 runs).

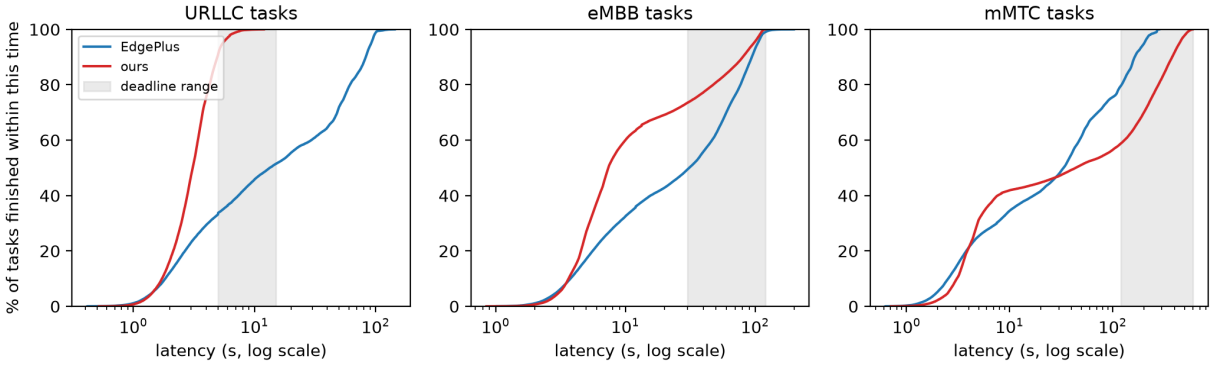


Figure 13: How quickly tasks of each class finish (CityPulse, one heavy-load test episode). A curve that rises further to the left means faster. The grey band is the class's deadline range.

6.8 Energy savings in detail

Figure 14 shows how the energy saving depends on load. The saving comes almost entirely from Rule 2 on CityPulse (dashed and solid blue lines overlap). Rule 2 slows down 97.8% of tasks at light load but only 21.9% at heavy load, because a busy server rarely has an empty queue. On AirDelhi the combination saves more than Rule 2 alone at medium and heavy load (5.5% against 2.9%, and 7.3% against 1.1%): Rule 1 also saves energy, because it does not spend CPU time on tasks that are already lost.

Figure 15 shows which speeds the rule actually picks. At light load most tasks run at 70–80% of full speed, and almost none at full speed. At heavy load, 74% of tasks still run at full speed. The rule naturally backs off when the city is busy.

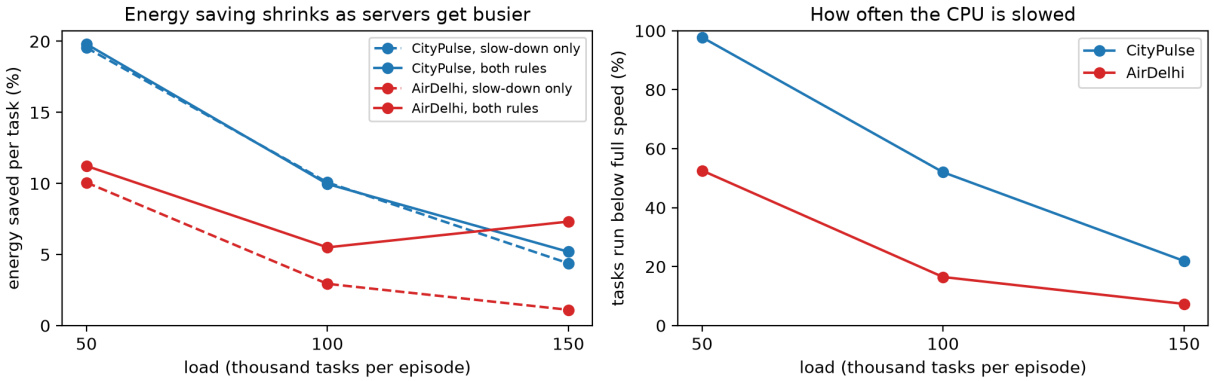


Figure 14: Left: energy saved per task compared with EdgePlus. Right: share of tasks run below full speed.

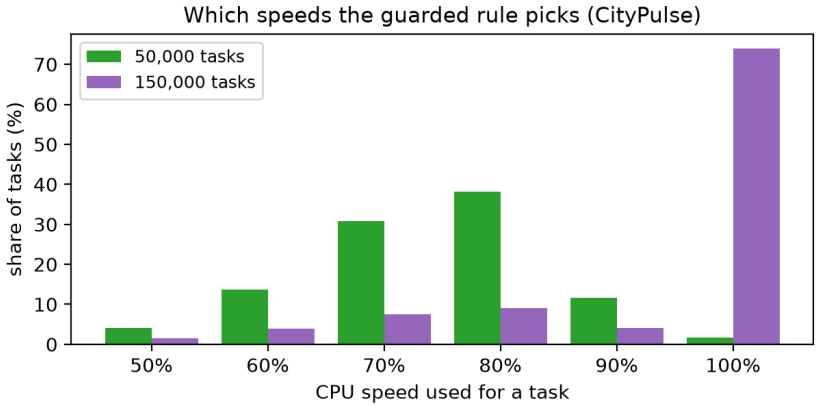


Figure 15: CPU speeds chosen by the guarded rule in one CityPulse test episode at light and heavy load.

6.9 Are the differences real?

Table 13 gives the paired statistical tests for our full method against EdgePlus. Every difference in on-time share and energy is statistically significant. The tests are worth reading carefully at light CityPulse load: there our rules lower the on-time share by 0.002 points, about two tasks in 100,000. The test correctly says this difference is real, but it is far too small to matter in practice.

Table 13: Paired tests of our full method against EdgePlus (30 pairs each, Wilcoxon signed-rank test, Benjamini–Hochberg adjusted p-values). A p-value below 0.05 means the difference is very unlikely to be luck.

Data	Load	On time		Energy per task		95th-pct latency	
		change	p	change	p	change	p
CityPulse	50,000	−0.002 pts	0.0005	−19.8%	<0.0001	+0.8 s	<0.0001
	100,000	+0.12 pts	<0.0001	−10.0%	<0.0001	−1.3 s	<0.0001
	150,000	+19.13 pts	<0.0001	−5.2%	<0.0001	+150.3 s	<0.0001
AirDelhi	50,000	+1.09 pts	<0.0001	−11.2%	<0.0001	+3.1 s	<0.0001
	100,000	+9.51 pts	<0.0001	−5.5%	<0.0001	+45.4 s	<0.0001
	150,000	+23.50 pts	<0.0001	−7.3%	<0.0001	−22.7 s	0.74

6.10 Do the rules need a learned scheduler?

Finally, we applied the same rules to the simple, non-learning scheduler from Section 5, which sends each task to the first server in routing order that can finish it in time. On CityPulse at heavy load the rules raise it from 72.0% to 98.8% on time, and at light load they cut its energy per task by 10.7%. So the rules help on their own, but the best results come from combining them with the learned EdgePlus policy (99.7% on time). The learned policy and the server rules solve different parts of the problem: one picks the right server, the other uses that server well.

7 Discussion

7.1 What this means in practice

1. **Choosing a good server is not enough.** Even a strong learned scheduler loses many urgent tasks if servers then work through their queues in arrival order. The order inside the server matters as much as the choice of server.

2. **Use deadlines as priorities, and drop lost causes.** Deadline order protects urgent tasks without starving relaxed ones, and dropping hopeless tasks frees time for tasks that can still be saved.

3. **Save energy only when it is safe.** Slowing the CPU is a cheap way to save energy, but only when no other work is waiting, and only by a bounded amount.
4. **Report results per class.** Overall numbers hid the fact that all of EdgePlus's misses were urgent and routine tasks (57% and 43% of the misses at heavy load), while relaxed tasks almost never missed.

7.2 Limitations

We want to be clear about what our results do and do not show.

- All results come from a simulator. We checked it against the EdgePlus paper, but the size of the gains depends on choices such as multi-core servers and the deadline ranges.
- Our EdgePlus rebuild leaves out PSO, the security layer and the original path planner, and uses three training seeds per load. At medium load it is much less congested than the original on CityPulse.
- CityPulse values are generated, not measured. AirDelhi is real, but we only tested models trained on CityPulse there; training directly on Delhi data might change the baseline.
- Dropping assumes a late result is worthless. If late logging results are still useful, dropping could be limited to URLLC and eMBB tasks.
- Our energy model uses a 25% static share and speeds down to half; it does not include idle power or sleep modes, which could save more at light load.
- Relaxed tasks wait longer. Applications that care about absolute delay rather than deadlines may not want this trade-off.

7.3 Future work

Natural next steps are to retrain EdgePlus with the rules switched on, so that it learns to make use of them; to train and test on real data from more cities; to add idle power and sleep modes to the energy model; and to bring back the PSO outer loop for a complete comparison with the original system.

8 Conclusion

EdgePlus decides which server should run each task in a 6G edge network, but treats all tasks the same once they reach a server. We added two simple rules inside each server: run the task with the earliest deadline first and drop tasks that can no longer make it, and slow the CPU down only when nothing else is waiting and only by a bounded amount. With the same trained EdgePlus model, these rules raise the share of tasks finished on time at heavy load from 80.5% to 99.7% on the CityPulse data and from 73.0% to 96.5% on real data from Delhi, and cut energy per task by up to 19.8% at light load. The price is a longer, but still on-time, wait for relaxed tasks. The experiments also show why each detail matters: deadline order works best together with dropping, deadlines make better priorities than fixed classes, and slowing down the CPU needs a guard.

Code and data availability

The simulator, the trained models, all raw results and the scripts that produce every table and figure in this paper are available in the project repository. CityPulse and AirDelhi are public datasets; AirDelhi is released under the CC BY 4.0 licence.

References

[1] B. H. Taher, J. Luo, F. Ghrabat, and Y. Qiao, "EdgePlus: A multiagent reinforcement learning framework for dynamic task allocation in 6G edge computing," *IEEE Internet of Things Journal*, vol. 13, no. 8, pp. 15 514–15 532, 2026.

[2] S. Chauhan, Z. B. Patel, S. Ranu, R. Sen, and N. Batra, "AirDelhi: Fine-grained spatio-temporal particulate matter dataset from Delhi for ML based modeling," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, vol. 36, 2023, pp. 75 455–75 468.

[3] B. K. Chaurasia, M. M. Shukla, V. Vishwakarma, and B. Tiwari, "Transformative impact of edge computing with 6G network using cloud computing," *Journal of Cloud Computing*, 2026.

[4] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[5] J. R. Jackson, "Scheduling a production line to minimize maximum tardiness," Management Science Research Project, University of California, Los Angeles, Research Report 43, 1955.

[6] C. D. Locke, "Best-effort decision making for real-time scheduling," Ph.D. dissertation, Carnegie Mellon University, 1986.

[7] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.

[8] M. Weiser, B. Welch, A. Demers, and S. Shenker, "Scheduling for reduced CPU energy," in *Proc. 1st USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 1994.

[9] P. Pillai and K. G. Shin, "Real-time dynamic voltage scaling for low-power embedded operating systems," in *Proc. 18th ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 89–102.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[11] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[12] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *Journal of the Royal Statistical Society: Series B*, vol. 57, no. 1, pp. 289–300, 1995.

[13] J. F. C. Kingman, "The single server queue in heavy traffic," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 57, no. 4, pp. 902–904, 1961.

[14] CityPulse Project, "CityPulse smart city datasets: Pollution measurements," University of Surrey, http://iot.ee.surrey.ac.uk:8080/datasets.html, accessed Nov. 2026.

[15] 3GPP, "Evolved universal terrestrial radio access (E-UTRA); further advancements for E-UTRA physical layer aspects," 3rd Generation Partnership Project, Technical Report TR 36.814, 2010.

[16] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. International Conference on Learning Representations (ICLR)*, 2016.

A All simulation parameters

Table 14: Parameters of the simulator and of training. Values marked † come from the EdgePlus paper.

Parameter	Value
City	10 km × 10 km†, 10 clusters†, weighted K-means and Delaunay neighbours†
Task classes	URLLC / eMBB / mMTC = 20 / 50 / 30%†; deadlines 5–15, 30–120, 120–600 s†
CPU work per task	averages 6 / 12 / 9 Gcycles; spread $\sigma = 0.47$ (log-normal); about 16% more per unit of severity
Data per task	2 / 8 / 1 KB; memory 64 / 128 / 32 MB
Loads	50,000 / 100,000 / 150,000 tasks in a 600 s window†
Radio	20 / 50 / 5 MHz per class; frequency reuse 3; 23 dBm power + 20 dB antenna gain; path loss $128.1 + 37.6 \log_{10} d$ (d in km) [15]; MAC efficiency 0.7; overhead 1.2
Slices	25 / 65 / 35% of a cluster's cores reserved per class over a 60 s window
Energy	2.2 / 2.8 / 1.8 mJ per Gcycle (stationary / mobile / cloud) at full speed; 25% static; device radio 0.2 W; 4 mJ per task sent to the cloud; 0.02 mJ per metre driven
Rule 2	speeds 50–100% in steps of 10%; at most 80% of the slack and 1.5 s extra
Ledger signals	reputation: moving average of on-time results (weight 0.05); threshold 0.6†; URLLC avoids servers below 0.4; debt target 0.9
Advisory	bonus weights: Nash 0.5, PID 0.5, leader 0.3; PID gains 1.90 / 1.20 / 0.06†; set point 80%; mobile servers move every 20 s
Reward	weights 1, 1, 0.2, 0.1, 0.2, 0.5, 0.5 for on time, late, latency, energy, cloud, reputation, debt; class weights 3 / 1.5 / 1
PPO	60 rounds; discount 0.95; GAE 0.9 [16]; clip 0.2; 4 passes per round; batches of 4096; learning rate 0.0003; entropy bonus 0.01
Networks	actor: 2 layers of 64 units; critic: 2 layers of 128 units; up to 16 candidate servers per decision
Tests	3 training seeds × 10 test episodes per load; models run greedily (always the best-scored server)